Selected Topics in Visual Recognition using Deep Learning

Homework – 1

Imrose Batterywala – 314540010

[github]

Introduction

This homework focused on an image classification task. The dataset was that of 100 classes, the training data was given to us, the test scores were computed online in a competetion benchmark. To solve this I created a deep learning pipeline which included data pre-processing, using a pre-trained ResNet as my backbone, making architectural changes to the model and finally hyperparameter tuning to improve performance.

Method

Data preprocessing: The data preprocessing pipeline first loads images from the folder-based dataset and converts all inputs to RGB format. Then, model-specific transforms are applied using timm, including resizing to 224 x 224, normalization with ImageNet mean and standard deviation, and several training-time augmentations such as RandAugment, horizontal and vertical flipping, color jitter, grayscale augmentation, Gaussian blur, and random erasing. In addition, the training pipeline uses a weighted sampler to reduce class imbalance, while MixUp, CutMix, and label smoothing are further applied to improve generalization. The main modification to tackle the class imabalance / long tailed dataset along with the poor split within the train and validation accurace was to combine them and make an even 9:1 split which represents each class evenly within train and validate respective to their class representation. Given in the image below are the default data augmentation settings.

```
parser.add_argument("--hflip", type=float, default=0.5)
parser.add_argument("--vflip", type=float, default=0.5)
parser.add_argument("--color-jitter", type=float, default=0.4)
parser.add_argument("--grayscale-prob", type=float, default=0.1)
parser.add_argument("--gaussian-blur-prob", type=float, default=0.1)
parser.add_argument("--random-erasing-prob", type=float, default=0.25)
parser.add_argument("--random-erasing-mode", type=str, default="pixel")
parser.add_argument("--random-erasing-count", type=int, default=1)
parser.add_argument("--label-smoothing", type=float, default=0.1)
parser.add_argument("--mixup-alpha", type=float, default=0.2)
parser.add_argument("--cutmix-alpha", type=float, default=1.0)
parser.add_argument("--mixup-prob", type=float, default=1.0)
parser.add_argument("--mixup-switch-prob", type=float, default=0.5)
parser.add_argument("--mixup-mode", choices=["batch", "pair", "elem"], default="batch")
```

Model architecture: The main model in this project is based on resnet152.a1h_in1k [1], which serves as the primary backbone for feature extraction and classification. In addition to this main model, I also trained two other smaller ResNet-based architectures initialized with pretrained weights. Using pretrained ResNet backbones helps the model start from strong visual representations learned on large-scale datasets, which improves convergence speed and classification performance on this task. Given below is the model card of the base model which I took from HuggingFace, it follows the project requirements of *< 100M parameters*.

Model card for resnet152.a1h_in1k

A ResNet-B image classification model.

This model features:

- ReLU activations
- single layer 7x7 convolution with pooling
- 1x1 convolution shortcut downsample

Trained on ImageNet-1k in `timm` using recipe template described below.

Recipe details:

- Based on ResNet Strikes Back `A1` recipe
- LAMB optimizer
- Stronger dropout, stochastic depth, and RandAugment than paper `A1` recipe
- Cosine LR schedule with warmup

Model Details

- **Model Type:** Image classification / feature backbone
- **Model Stats:**
  - Params (M): 60.2
  - GMACs: 11.6
  - Activations (M): 22.6
  - Image size: train = 224 x 224, test = 288 x 288

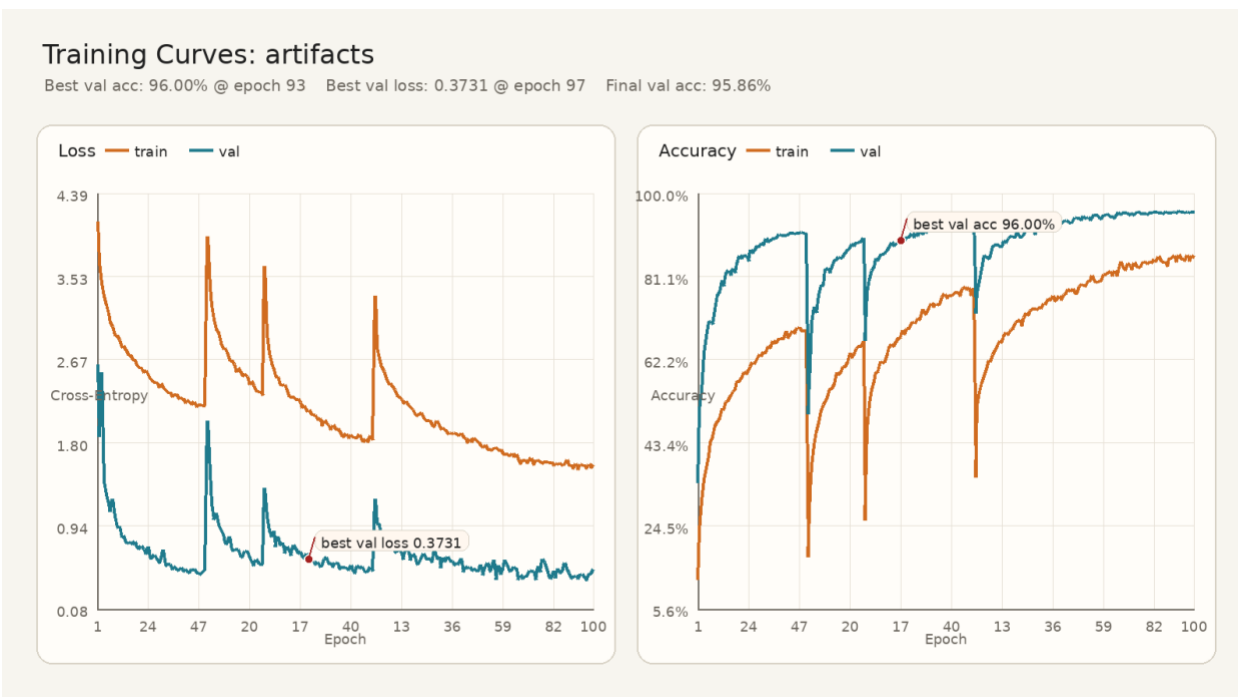
Hyperparameter Setting: The model was trained with a batch size of 8 and optimized using AdamW with a learning rate of 3e-4 and weight decay of 1e-4. The training process used 100 epochs with a cosine annealing learning rate scheduler. For regularization and robustness, label smoothing was set to 0.1, while MixUp and CutMix were applied with mixup_alpha = 0.2 and cutmix_alpha = 1.0. Given below are the default hyperparameters.

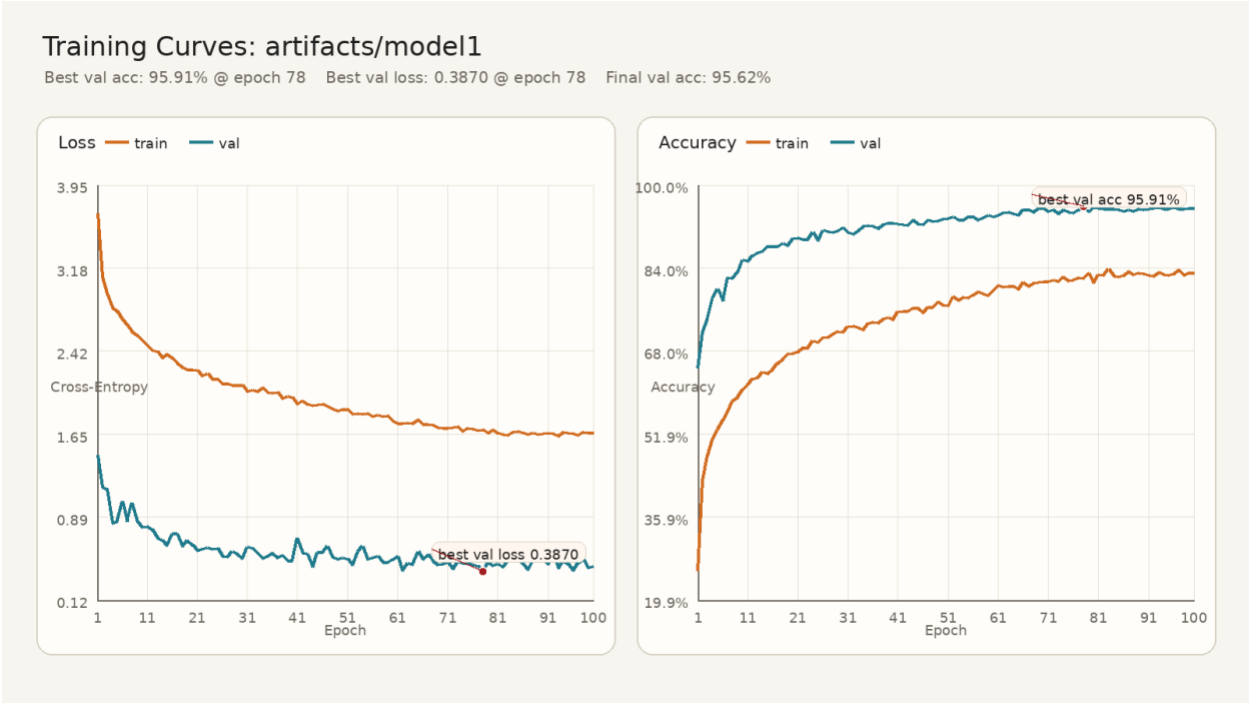
```
parser.add_argument("--epochs", type=int, default=50)
parser.add_argument("--batch-size", type=int, default=8)
parser.add_argument("--num-workers", type=int, default=4)
parser.add_argument("--lr", type=float, default=3e-4)
parser.add_argument("--weight-decay", type=float, default=1e-4)
parser.add_argument("--seed", type=int, default=42)
```

Further Modifications: Beyond the baseline pretrained backbone, I introduced several modifications to improve generalization and handle the dataset more effectively. These include stronger data augmentation, weighted sampling to address class imbalance, and label-mixing strategies such as MixUp and CutMix. I also trained multiple models and combined them during inference, which helps reduce model variance and leads to more stable predictions than relying on a single network.

Results

Training Curve: Below is the training curve of the base model and one of the supporting models





Confusion Matrix: Below is the confusion matrix of the base model.

These are just some training / final validation results from the process.

Additional Experiments

I added SoftTargetCrossEntropy because MixUp/CutMix gives soft labels, so normal hard-label cross entropy does not really match the training target. This should make the optimization more consistent. I also trained two smaller

ResNet models with slightly different settings so they would make different kinds of errors. Each one alone was weaker than the best single model, but with validation-accuracy-weighted along with TTA ensemble they could complement each other and push the validation accuracy from 96.00% to 96.38%. So the gain seems to come more from using the correct loss and model diversity, not just hyper-parameter tuning.

```
class SoftTargetCrossEntropy(nn.Module):
    def forward(self, logits: torch.Tensor, targets: torch.Tensor) -> torch.Tensor:
        return torch.sum(-targets * F.log_softmax(logits, dim=-1), dim=-1).mean()
```

Hypothesis: This modification was done because the augmentation generates soft labels, the main reasoning being that training the model against label distribution over one-hot encoding will regularize and improve model generalization.

Potential pitfall: The loss in training works by matching the model's log-probabilities to blended target probabilities, but it may help less or even hurt if label mixing makes samples too ambiguous or washes out class-specific features.

Results: There was an improvement when the test results were uploaded on the leaderboard.

References

1. Wightman, R., Touvron, H., Jégou, H. (2021). ResNet strikes back: An improved training procedure in timm. arXiv preprint arXiv:2110.00476.